

Build a Quiz Web App with Agentic Coding

Prompt Engineering — Practical Assignment

Estimated time: 3–4 hours · Tool: Claude Code · Deliverable: a working web app + git repo

The one rule: all code must be written by Claude Code. You may not write or edit code by hand. Your job is to direct, review, and iterate.

Learning Objectives

By the end of this assignment you should be able to:

- Set up a project for agentic coding, starting from a well-formed CLAUDE.md.
- Decompose a feature set into small, sequential prompts instead of one giant request.
- Write specific, testable instructions and recognise vague ones.
- Review generated code, catch problems, and give corrective feedback that lands.
- Use hooks and subagents to make the agent check its own work.

The Build: Quiz Web App

Core Requirements (must have)

- **Quiz flow** — show one question at a time with 4 answer buttons; advance on click.
- **Scoring** — track points per question and show a running total.
- **Timer** — 15-second countdown per question; auto-skip when time runs out.
- **Results screen** — final score, accuracy %, and time taken.

Gamification (pick at least 2)

- **Streak bonus** — 2× points after 3 correct in a row, 3× after 5.
- **Leaderboard** — save the top 10 scores in localStorage and show them after each game.
- **Difficulty levels** — easy (20s), medium (15s), hard (10s).
- **Progress bar** — a visual indicator of how many questions remain.
- **Animations** — correct/wrong feedback — colour flash, shake, confetti.
- **Sound effects** — use the Web Audio API for correct, wrong, and timer sounds.

Bonus (optional)

- Add 10+ questions on any topic you like.
- Add categories (e.g. Claude Code, game design, general knowledge).

- **Multiplayer** — two players take turns on the same screen.
- **Custom theme** — match your branding.

Workflow: Practice What You Learned

This is an **agentic coding exercise**, not a coding exercise. How you work matters as much as what you ship. Follow this workflow:

- **Start with CLAUDE.md** — run `/init`, then edit it to add build commands, conventions, and a NEVER list.
- **One feature at a time** — don't overload Claude with five features in one prompt. Build the quiz flow first, then layer the rest on.
- **Be specific** — "add a 15-second timer that shows a countdown bar and auto-skips on timeout" beats "add a timer."
- **Set up hooks** — auto-format and auto-test after every edit so Claude checks its own work.
- **Review and refine** — test in the browser after each feature; when something breaks, tell Claude exactly what's wrong.
- **Use subagents** — when you need an audit or want to explore multiple approaches without polluting your main context.

Grading Rubric

Criteria	Points
Quiz flow works (show → answer → next → results)	25
Scoring is correct	15
Timer works with auto-skip	15
At least 2 gamification features	20
CLAUDE.md exists and is useful	10
Code is clean (no console errors, responsive layout)	10
Creativity bonus (extra features, polish, fun factor)	5
Total	100

Submission

Push your project to a personal git repo and share the link. Include a short note (2–3 sentences) on what was easy and what was hard about directing Claude Code.

Make sure your repo includes your **CLAUDE.md** — it's part of what we grade, and it's the clearest signal of how you set the agent up for success.